

1 Токенизация

Нейронные сети не способны обрабатывать текстовые данные в “сыром” виде, поскольку спроектированы для работы с числовыми тензорами. Текст представлен последовательностью символов, таких как буквы, цифры, пробелы, знаки пунктуации и специальные символы. Какие-то из этих элементов встречаются чаще, какие-то — реже. Сами тексты, очевидно, могут быть разной длины.

Процесс разбиения текста на последовательность элементов называется **токенизацией**, а сами эти элементы — токенами. Все уникальные токены сводятся в словарь, где им присваивается собственный идентификатор. Это обеспечивает важное свойство токенизации — обратимость. Благодаря этому свойству, исходный текст в неизменном виде может быть декодирован из последовательности токенов.

1.1 Посимвольная токенизация

Самой первой идеей, которая приходит в голову, как разделить текст на последовательность, является посимвольное разбиение. Исходный текст итак можно представить в виде последовательности идентификаторов. Например, мы можем взять таблицу кодировки utf как словарь.

Поскольку уникальных символов немного (за исключением некоторых языков), размер итогового словаря будет небольшой, что благоприятно отразится на размере модели. Однако, если такой подход и будет успешен при обработке иероглифических языков, таких как китайский, японский или, например, древнешумерский, то при работе с другими языками могут возникнуть проблемы. При разбиении текста на символы, количество токенов может быть в десяток раз больше, чем количество слов. Также, стоит учитывать, что в отличие от китайского иероглифа, буква латинского или кириллического алфавита почти не несёт семантической (то есть смысловой) ценности.

1.2 Токенизация по словам

Посимвольная токенизация приводит к очень большой длине входной последовательности. Как решить эту проблему? Попробуем разбивать текст не по символам, а по словам. Хотя это решение и значительно сократит размер последовательности, а каждый токен будет иметь большую семантическую ценность, из-за большого количества различных форм слов, а также множества редких слов, размер словаря может раздуться до миллионов значений.

Конечно, редкие слова можно заменять одним специальным токеном, предназначенным для неизвестных элементов. Однако, это мало того, что всё ещё не решает проблему огромного словаря, так и при обработке текста может потеряться смысл этих редких слов.

1.3 Byte-pair encoding (BPE)

Очевидно, что для эффективной обработки текста необходим компромиссный вариант между посимвольной токенизацией и токенизацией по словам, сочетающий преимущества обоих вариантов и минимизирующий их недостатки. Одним из таких компромиссных вариантов стал **ВРЕ-токенизатор**.

Мы хотим, чтобы часто повторяющиеся части слов были одним токеном. Возьмём эту идею за основу. Сначала разобьём каждое слово на символы и добавим сепаратор конца слова. Словарь в начале состоит из символов стартового алфавита, сепаратора конца слова и специальных токенов: <PAD>, <BOS>, <EOS>, <UNK>.

Далее, на большом корпусе текста необходимо найти самую частую пару соседних токенов. Например, для текста

Byte-pair encoding tokenization

если разбить слова на символы и добавить сепаратор '</w>', то получим пары соседних токенов:

'By', 'yt', 'te', 'e-', '-p', 'pa', 'ai', 'ir', 'r</w>';
'en', 'nc', 'co', 'od', 'di', 'in', 'ng', 'g</w>';
'to', 'ok', 'ke', 'en', 'ni', 'iz', 'za', 'at', 'ti', 'io', 'on', 'n<w>'.

Пара 'en' встретится 2 раза, а остальные — по 1 разу. Поэтому добавим токен 'en' в словарь и запомним merge-правило, после чего повторим эту последовательность шагов до того, как размер словаря не достигнет желаемого. При запоминании merge-правил крайне важно учитывать их последовательность.

Предположим, что мы обучили ВРЕ-токенизатор, но как с его помощью токенизировать текст? Добавленные элементы начального словаря определённо будут подсловами токенов в словаре. Да и неизвестно, какого размера будет каждый токен, чтобы найти его. Для этого мы и запоминали merge-правила. Чтобы токенизировать текст, применим merge-правила по порядку, в котором их сохраняли.

Алгоритм обучения и токенизации может показаться очень долгим. Однако, токенизацию текста можно применять параллельно по участкам текста, например по предложениям. Обучение ВРЕ также можно частично распараллелить: на разных участках корпуса параллельно подсчитывать частоты пар, а затем складывать их. Само обучение остается итеративным, поскольку после каждого merge-правила частоты пар меняются.

ВРЕ-токенизация предлагает компромисс между посимвольной токенизацией и токенизацией по словам, разбивая текст на последовательность подслов. В свою очередь оказывается, что такие подслова часто являются морфемами: приставками, суффиксами, корнями, окончаниями слов. Также, ВРЕ-токенизатор способен запоминать имена и названия как отдельные токены.

Byte-level ВРЕ токенизация

Пусть мы хотим обучить ВРЕ-токенизатор для русского языка. Для этого

взяли корпус сочинений Льва Николаевича Толстого (предварительно удалив отрывки на французском языке). Однако, когда модель с обученным токенизатором будет использована в чат-боте, то она может столкнуться с неожиданным входом: множество эмодзи и особые символы. А что делать, если мы хотим обучить мультязычный токенизатор?

Byte-level BPE токенизатор решает проблему тем, что сначала переводит текст в байты и работает уже не с набором символов, а с набором байтов. Пробел кодируется одним байтом. Такой токенизатор подойдёт как для кириллицы, так и для латиницы, эмодзи и даже бинарных файлов.

1.4 Embedding-слой

После токенизации, заменим токены в тексте на их индексы. Технически, уже эту последовательность можно обрабатывать нейронной сетью. Однако, идентификаторы токенов обладают “псевдопорядком”, из-за этого, например, токен ‘tok’ с номером 29 будет ближе к токenu ‘byte</w>’ с номером 28, чем к токenu ‘en</w>’ с номером 239. Хотя вряд ли можно говорить о какой-то их особой смысловой близости, или, тем более, о том, что один токен почему-то больше другого.

При работе с табличными данными для обработки качественных признаков часто используется one-hot-encoding. Но в текущей ситуации он вряд ли применим, поскольку потребует создания гигантской квадратной матрицы с количеством строк и столбцов равными длине словаря. К тому же, эта матрица будет крайне разреженной, то есть состоять в основном из нулей, с редкими единицами.

Вместо разреженной матрицы попробуем обучить плотную матрицу — **embedding-слой**. В этой матрице вектор соответствует тому токenu, чей индекс равен индексу этого вектора. Эти вектора (а вернее весь embedding-слой) инициализируются случайным образом и обучаются вместе с остальными параметрами модели.

Приятным бонусом embedding-слоя является то, что токены, часто встречающиеся в похожих контекстах, например ‘король’ и ‘монарх’, нередко имеют близкие вектора — то есть угол между ними будет мал. Из-за этого возможна арифметика векторов. Например, если из токена ‘король’ вычесть токен ‘мужчина’ и добавить токен ‘женщина’, то получится токен, очень близкий к токenu ‘королева’. Другими словами, embedding-слой отображает токены в **семантическое пространство**.